

Exactly-Once Effects for Regrouping Agent Tool Calls:

A Reduction, a Retention Hazard, and an Evidence Contract

Research-team working draft

Version 0.3 — 19 September 2026

Development results; not submitted; authorship confirmation pending

Abstract—A recovering language-model agent may re-express an authorized task with different tool calls. The runtime must decide which authorized effect occurrences remain available, not whether the new request text equals the old one. We study a restricted language of independent, explicitly authorized effect obligations executed against a durable sink. Three results follow. (i) *Reduction*. With a sink that atomically retains a per-key effect record for the whole recovery period, verified plan refinement reduces to stable obligation-key execution: a second implementation written from the v0.2 specification reproduces the v0.2 null result exactly across 186,674 exhaustively enumerated regrouping cases, 240 real SIGKILL/recovery trials and 24 three-worker races, and a reference that consults its own ledger dispatches the same set as the verifier by construction. (ii) *Retention hazard*. The two commercial idempotency contracts we examined are finite: Stripe prunes idempotency keys after at least 24 hours and treats a reused key as a new request; Amazon SQS FIFO deduplicates for 5 minutes. We model retention-bounded sinks $P_D(T)$, prove that same-key retry is safe against every history the controller cannot rule out exactly when its own hold timestamp is within the window less a stated end-to-end timing bound, and show that every same-key-retry policy—including the strong reference and the v0.2 verifier—duplicates in every lost-acknowledgement case once the window closes (149,217 of 186,674 enumerated cases, a seven-step model-checking witness, and 20/20 runtime duplicates for naive recovery versus 0/20 for the retention-aware controller). A lookup-then-fresh-key recovery without a fence—a hand-written handler, not the same-key retry a durable-execution engine prescribes—duplicates whenever the old request is still in flight (15/15 in-flight takeover trials, on every profile including the deduplicating one) and is correct whenever it has landed (0/15). Completion after the window then depends on an evidence contract: the retention-aware controller completed 10/10 lost-acknowledgement trials with a lookup endpoint and left 10/10 unresolved without one. (iii) *Evidence contract*. A ternary per-obligation projection is sufficient for the maximal safe fresh-dispatch set but not for reconciliation cost: retaining the processing class of a lost batch request changes the exact optimal expected number of per-item evidence calls from n (independent entries) to 1 (atomic) and to about $\log_2(n+1)$ (ordered stop-on-failure) when finality is provided per request, and to 4.5 and 4.9 of 8 when every fresh dispatch must be preceded by a per-key fence and the policy minimises that total; a journal-driven recovery worker reproduces these counts on the runtime, and a misdeclared class produces false completion. A takeover study across three provider profiles shows where a fence, an evidence lookup and an explicit unresolved state each do work

the others do not, and a resumed-predecessor study shows that a stale worker’s receipt is evidence the controller can adopt without weakening its lease rule. No live LLM, commercial provider call, universal exactly-once guarantee or superiority over published systems is claimed.

Index Terms—agent systems, effect identity, crash recovery, idempotency, retention, authorization, partial observability

I. INTRODUCTION

An agent instructed to send one deployment notification to each of two recipients can use one batch call, two single-recipient calls, or a different ordering. After a crash it may choose another representation. Asking whether the new request has the same text as the old one does not specify the recovery decision. The runtime must determine which authorized *occurrences* are being attempted and what is known about their previous effects.

Three ambiguities must be separated. Identical payloads can belong to different approvals: a user may legitimately request the same notification again. One approval can produce different request representations without authorizing additional effects. And a request may have committed even though its result is absent from the checkpoint. Existing work addresses parts of this space: semantic rollback attacks and replay-or-fork recovery [1], durable authorization instances bound to canonical actions [2], reservations under ambiguous provider outcomes [3], and task-level staged effects [4]. Durable-execution engines recommend that side-effecting steps be idempotent and restart each attempt from its initial state [7]; completion-record systems established durable request identity and atomic effect/result recording [5].

The v0.2 study began from the hypothesis that preserving effect obligations across plan changes would add safety or completion beyond a strong durable-workflow reference, and found a null result. This version asks the follow-on question that the null result forces: *under which provider contracts does stable-key execution stop being sufficient, and what must a controller then know?* Two answers turn out to be concrete and testable. First, the sink assumption behind the reduction—retention of the key/effect record throughout recovery—is not what commercial providers document. Stripe removes

idempotency keys after they are at least 24 hours old and “generate[s] a new request if a key is reused after the original is pruned” [8]; Amazon SQS FIFO deduplication lasts five minutes [9]. A recovery that lands after the window is a retry into a sink that has forgotten the key. Second, when the sink no longer deduplicates, safe completion depends on outcome evidence, whose cost depends on what the controller’s journal retained about the lost request.

Contributions:

- 1) A second implementation of the v0.2 kernel written from its specification by the same research process, reproducing every finite-domain count (Table I) and the process-crash and concurrency results. It is not an external replication. The reference is strengthened to consult its own ledger (R^+); Lemma 1 shows it dispatches the same set as the verifier, so the tie is structural rather than empirical.
- 2) A retention-bounded profile $P_D(T)$ with a condition for safe same-key retry that depends only on the controller’s hold timestamp, the documented window and one end-to-end timing bound, necessary and sufficient when every attempt commits or dies within a lifetime shorter than the window, under an explicit adversarial timing model (Prop. 3); the exhaustive enumeration in Table II, whose counts are a corollary of the proposition and serve as an implementation check; a model-checking mutant with a seven-step witness; and a runtime study with a finite-TTL sink (Table VI).
- 3) A takeover study in which a successor worker adopts a held occurrence while the crashed worker’s request is still in flight, across P_D , P_F and P_O with three recovery policies: naive same-key retry (what a durable-execution engine does when the idempotency key is the activity identity), lookup-then-fresh-key without a fence (a hand-written handler), and profile-aware recovery (Table IV). The middle policy is the realistic baseline; it fails exactly in the in-flight case. A resumed-predecessor study (Table V) shows that the ledger’s lease-holder rule protects controller state but not the effect count, and that under P_O the resumed worker carries the only finality evidence, which a ledger can adopt safely by distinguishing evidence binding from lease ownership.
- 4) A semantics-determined belief construction: recording the membership, order and processing class of a lost batch request determines the belief family for the atomic, prefix and independent classes; ternary projection discards it. Closed forms for the atomic and prefix classes are verified against an exact adaptive oracle to $n = 10$, and the independent class to $n = 8$, under two evidence-cost accountings; the per-key accounting minimises total fences rather than probe count (Table VII); a recovery worker that rebuilds the belief family from a one-field-per-request journal reproduces the expected counts on the runtime and a misdeclared class yields false completion (Table VIII).

- 5) A measured negative finding about a common evaluation design: three-worker races resolved by controller reservation never reach the sink’s deduplication path (0 of 96 sink calls), so they do not test the sink contract.

The empirical scope remains explicit. Proposals are generated by combinatorial transformations, not by a live LLM. The provider performs synthetic database insertions. Finite enumerations are exhaustive only in the stated models; process schedules are sampled. Commercial contract facts are quoted from official documentation, not exercised against a live account.

II. BACKGROUND AND PRIOR-ART BOUNDARY

A. Durability and completion records

Idempotent, uniquely identified messages are the classical answer to at-least-once delivery across trust boundaries [16], [17]; sagas compensate rather than deduplicate and require the compensating action to be known [18], which an agent’s regrouped tool call typically is not. RIFL retains completion records with operations and addresses retry routing and record lifetime [5]. A persistent identifier alone is insufficient if the external mutation and the deduplication record can diverge, if retries reach non-coordinating sinks, or if the record is reclaimed. RIFL treats record lifetime as a system-controlled parameter; a commercial provider controls it unilaterally, which is the gap this version studies. Stream processors obtain exactly-once by making the sink participate in a transaction or a checkpoint epoch [19], [20]; an agent’s effect provider offers neither, only a documented key contract. The HTTP Idempotency-Key draft standardizes the client-supplied key and the fingerprint check but explicitly leaves the retention period to the server [21].

CapLease binds authorization instances to canonical actions with durable budgets and transactional transitions; its Server Ledger reference prevents duplicate effects with an idempotent sink [2]. A result against regenerated keys therefore does not establish an advantage over that design; we do not execute CapLease.

B. Agent-specific execution boundaries

ACRFence analyzes regenerated calls and either reuses an old result or requires a fork [1]. AID-Guard holds a reservation while an outcome is ambiguous and permits replacement only under terminal-result or no-effect/fencing conditions [3]; our fence-then-fresh-key path follows that boundary. Cordon introduces task-level semantic transactions with result lineage and staged effects [4]; we do not claim all-or-nothing commit across obligations. Proof-carrying agent actions associate verifiable evidence with proposals [6]; our checker is an exact contract check, not a certificate system. Tool-using agents in the ReAct and Toolformer families interleave reasoning with tool calls whose text is regenerated on every step [22], [23]; that regeneration is the source of the regrouping we study, and none of these systems addresses the identity of the effect across a crash.

C. Durable execution and commercial idempotency

At-most-once delivery with a bounded duplicate cache is classical: Birrell and Nelson’s RPC [25], Lampson’s account of reliable messages [26], and the synchronized-clock protocol of Liskov, Shrira and Wroclawski, which detects a duplicate after per-sender state has been discarded provided clocks stay within a known bound [27]. Leases [28] and fencing tokens such as Chubby’s sequencer [29] are the mechanisms behind our lease-holder rule and the P_F fence. Workflow shims that make a non-idempotent service idempotent [30], intent locks [31], and exactly-once serverless logs [32], [33] assume the shim can see the effect; an agent’s tool provider generally cannot be wrapped that way. Durable Functions give a semantics for restarting an orchestration from its log [34]. Temporal documents that an Activity “be idempotent, so retries can be processed without duplicate side effects” and that “each attempt starts from the initial state” [7]. Deduplication is thus delegated to the effect provider, and a retry under the activity’s own key is our *naive* policy. Prop. 3 has the same shape as the synchronized-clock bound, with two differences that matter here: after expiry those protocols reject the duplicate, whereas Stripe accepts the pruned key as a new request, and T is a vendor parameter the controller does not choose. Stripe saves the result of the first request under an idempotency key, compares incoming parameters to the original and errors on mismatch, and removes keys after at least 24 hours, after which a reused key produces a new request [8]. SQS FIFO deduplicates by `MessageDeduplicationId` within a 5-minute window and continues tracking the identifier after delivery [9]. These are the only two documented windows we rely on; both are finite.

D. Partial outcomes and evidence contracts

SQS `SendMessageBatch` reports each entry individually and can mix successes and failures under HTTP 200 [10]; SES `SendBulkEmail` returns “one object per intended recipient” [11]. These are *independent* per-entry semantics. Slack’s `conversations.history` provides message evidence after the fact, but for some application classes it is rate limited to one request per minute with at most 15 objects [12]; and Slack’s write method documents that on `fatal_error` “it’s possible some aspect of the operation succeeded before the error was raised” [13], which is the opaque-outcome ambiguity in a vendor’s own words. Planning with partial information and sensing is established [14], [15]; our belief families are an application of that view with a set-valued, prior-uniform belief and unit-cost sensing, and our finite-state exploration follows the explicit-state model-checking tradition [24] rather than proving the protocol in a theorem prover. Optimal decision trees are NP-complete in general [35], which is why the probe oracle is exponential and the closed forms are per class.

III. PROBLEM MODEL

A. Three identities

An immutable approval contract is $C = (\iota, p, \alpha, \Omega, s, v)$: intent-instance identifier, principal, authenticated approval

event, finite set of obligation occurrence identifiers, approved effect specification per obligation, and schema version. An *attempt* is a proposed or transmitted implementation of one or more obligations. An obligation identifies an occurrence, not a payload equivalence class; two identical notifications approved twice are two occurrences. The trusted approval layer creates ι and Ω ; re-presenting the same (p, α) recovers the same contract.

The executable language supports exact tuples (kind, target, body) with kind `notify`, a singleton wrapper and a batch wrapper. Aliases, free-text inference, decomposition, cross-provider equivalence and state-dependent transformation are excluded. Obligations are independent.

Proposition 1 (Content-only ambiguity). *A decision procedure observing only old and new payloads cannot always distinguish replay under one approval from an identical action under a new approval.*

Proof. Completed x , lost receipt, repeated proposal yields observed input (x, x) ; completed x followed by a new approval for x yields the same input. The required decisions differ. Approval provenance is the missing observation. $\square$

B. Failure and trust assumptions

The proposer may reorder, regroup, omit, duplicate or alter proposed entries. The verifier rejects unauthorized entries before dispatching any prefix. Workers stop abruptly; concurrent workers may attempt the same obligations; a successor may adopt a held occurrence while the predecessor’s request is still in flight. Conversation rollback never rolls back the contract, the control ledger or the provider ledger. Excluded: malicious host, corruption of both stores, lost disks, broken database atomicity, hostile clock manipulation beyond a bounded skew.

C. Provider profiles, now with retention

Definition 1 (Profiles). $P_D(T)$: *atomic per-key effect/result recording with immutable key/payload binding, retained for at least T after the first commit under the key; after T the key may be reclaimed and a request under it may commit again.* $P_D = P_D(T)$ with $T = \infty$. P_F : *no key deduplication, but a fence operation that prevents any later delivery under a key and returns a trustworthy final outcome for it.* P_O : *neither; lookups are point-in-time only.*

Stripe’s documented behaviour is $P_D(T)$ with $T \geq 24$ h and immutable parameter binding [8]; SQS FIFO is $P_D(T)$ with $T = 5$ min [9]. No profile is silently promoted to another; in particular a negative lookup alone never establishes P_F .

IV. VERIFIED REFINEMENT AND ITS REDUCTION

A. Plan checking and stable keys

A plan contains an instance, schema and a sequence of wrapper groups. Verification checks the whole proposal: certified wrappers, legal arity, each slot approved and used at most once, and exact match of kind, target and body against the contract. Executable bytes are reconstructed from the

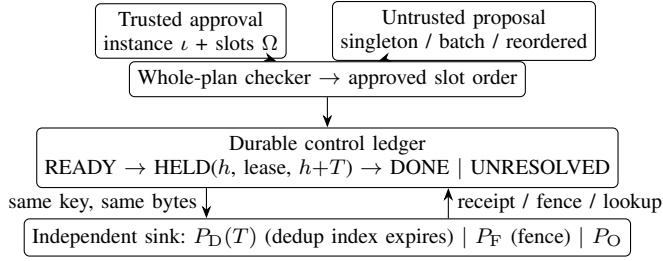


Fig. 1. Implemented boundary. The sink effect is a synthetic database insertion. Recovery of a HELD occurrence consults $h(\omega)$, T and the profile before choosing same-key retry, fence, evidence lookup or UNRESOLVED.

contract. Partial plans are accepted; completed occurrences in a recovered plan are residualized and return cached completion.

The effect key is $K(\iota, \omega) = H(\text{obligation-key-v1}, \iota, \omega)$, independent of grouping and attempt identity. Each occurrence has durable state *ready* $\rightarrow$ *held* $\rightarrow$ *done*; this version adds an *unresolved* state, quiescent for the controller’s own actions, for a held occurrence that can no longer be retried safely and for which the holder has no finality evidence. A control transaction reserves the occurrence (with a lease and, for $P_D(T)$, a retention deadline $h(\omega) + T$, where $h(\omega)$ is the controller time of this reservation) before transport.

Proposition 2 (Stable-key reduction). *Assume immutable contracts, complete plan checking, mediated effect paths, an injective persistent key map, and $P_D(T)$ with every attempt under a key, the original included, committing before $h(\omega) + T$. Then any accepted sequence of regrouped plans and retries has at most one committed effect per approved occurrence and no effect outside the contract.*

Proof. Every dispatched effect derives from an approved occurrence and its stored specification. All attempts for that occurrence use the same key. While the dedup record is retained, atomic key/effect recording allows at most one effect per key and rejects payload substitution. The retention hypothesis guarantees retention at every retry. Distinct occurrences are not collapsed by key identity. Induction over sink operations preserves the invariant. $\square$

Lemma 1 (Structural tie). *Let R^+ validate the whole action list before dispatching any of it, then perform canonical-action matching per occurrence, skip occurrences recorded done in the ledger and dispatch the rest under $K(\iota, \omega)$; let V check the whole plan, residualize against the same done set and dispatch the rest under the same keys. On any accepted destination plan both dispatch exactly the set of approved occurrences not recorded done, in the destination order, with identical bytes; hence identical sink calls and effect vectors.*

Proof. Both filters compute “approved and not done” per occurrence. Because R^+ validates the whole list first, a rejected plan dispatches nothing in either policy; the checkers differ only on those rejected plans. $\square$

The v0.2 reference R resent every occurrence and let the sink deduplicate, so residualization saved calls without changing effects. In this artifact the abstract R^+ and V share one code path and the runtime policy labeled R in the process studies is this ledger-consulting R^+ , not the resend-everything reference, so the equal counts in Table I check the implementation rather than test a hypothesis. Verified refinement adds no exactly-once guarantee under the retention hypothesis; the lemma is the null hypothesis for every comparison below.

V. FINITE RETENTION

A. The hazard

The hypothesis “every retry completes before $h(\omega) + T$ ” is the load-bearing assumption of Prop. 2, and it is a property of the recovery *schedule*, not of the code. A worker crash followed by an outage longer than T —or a checkpoint restored days later—violates it. The controller then holds an occurrence in state *held* whose effect may or may not have committed; the sink has forgotten the key; and a same-key retry is indistinguishable, at the sink, from a first request.

Proposition 3 (Retention-bounded same-key retry). *Let the controller record $h(\omega)$ durably before transport and issue a same-key retry at controller time $t \geq h(\omega)$. Let c be the earliest sink time at which the original attempt could commit and a the sink time at which the retry arrives, and let the end-to-end timing error $\varepsilon = (a - c) - (t - h(\omega))$ satisfy $|\varepsilon| \leq \Delta$ in every admissible history. Let the attempt lifetime be L : the original either commits at a sink time c' with $c \leq c' \leq c + L$, or never commits. The adversary may choose any such history. On a $P_D(T)$ sink with $L + \Delta < T$, the retry is duplicate-free in every admissible history if and only if $t - h(\omega) < T - \Delta$. If $L + \Delta \geq T$, no such t exists.*

Proof. (If) Assume $L + \Delta < T$ and $t - h < T - \Delta$. If the original has committed at c' before the retry arrives, its record lasts until at least $c' + T \geq c + T$, while $a \leq c + (t - h) + \Delta < c + T$, so the retry is deduplicated. If it has not, the retry is the first commit and its record lasts until at least $a + T$. The original can still commit only at some $c' \leq c + L$. Since $a \geq c + (t - h) - \Delta \geq c - \Delta$ and $L < T - \Delta$, one has $c' \leq c + L < c + T - \Delta \leq a + T$, so that late commit meets the retry’s record and is deduplicated too. (Only if) If $t - h \geq T - \Delta$, the adversary commits the original at exactly c , which any $L \geq 0$ allows, and chooses $\varepsilon = \Delta$; then $a \geq c + T$, the record may have been reclaimed, and the retry commits a second effect. If instead $L + \Delta \geq T$, the adversary lets the retry commit first and lands the original at $c + L$ after the retry’s record expires, for every t ; that is the in-flight duplicate of Table IV moved onto a short window. $\square$

For clocks with a constant offset, Δ is the maximum sink-side delivery delay of the retry relative to the original; if the offset can change between hold and retry, Δ additionally includes twice the skew bound. L is a provider fact too: an attempt the provider can still execute after the retention window is not covered by same-key retry at any t . The

documented windows we use, at least 24 hours and 5 minutes, sit well above a request's network lifetime, which is why the regime $L + \Delta < T$ is the one that applies to them; we did not measure L . The studies below use $\Delta = 0$ on one machine, so they exercise the window, not the margin.

The decision uses only the controller's own ledger and two contract parameters, T and Δ . No sink query can substitute for it: a negative lookup after reclamation is consistent with both histories.

Corollary 1 (Profile downgrade). *After $h(\omega) + T - \Delta$, a key on a $P_D(T)$ sink is no longer deduplicated. For that key the sink is then at best P_F , if it exposes a fence or another finality operation, and P_O otherwise.*

Definition 2 (Retention-aware recovery). *Inside the bound of Prop. 3 the controller retries the same key. After it, positive point-in-time evidence confirms; a fence permits a fresh key; a negative point-in-time lookup establishes nothing, so absent a fence the occurrence becomes unresolved.*

Where the objects a request created remain retrievable after the key is pruned—as Stripe's list endpoints permit for created objects, a capability we did not exercise here—the sink degrades to a lookup-capable profile; SQS offers no message lookup by deduplication identifier, so it degrades to P_O . The controller must therefore carry T and the evidence capability as explicit provider-contract fields.

B. Ambiguous replacement

A fresh key is different from a same-key retry. Certifying absence from a negative lookup and then dispatching a fresh key permits two distinct keys to commit one obligation if the old request can still arrive. Safe replacement in P_F needs an operation that certifies no old effect and prevents old delivery [3]; Section IX-D exercises this with a real in-flight request.

VI. OUTCOME KNOWLEDGE BEYOND INDEPENDENT STATES

A. Safe fresh-dispatch frontier

Under P_F , let a nonempty belief family $\mathcal{B} \subseteq 2^\Omega$ contain every finalized committed set consistent with sound evidence. Define $P(\mathcal{B}) = \bigcup \mathcal{B}$, $D(\mathcal{B}) = \bigcap \mathcal{B}$, $F(\mathcal{B}) = \Omega \setminus P(\mathcal{B})$ and $U(\mathcal{B}) = P(\mathcal{B}) \setminus D(\mathcal{B})$.

Proposition 4 (Maximal safe fresh dispatch). *A once-only fresh dispatch set S is duplicate-free in every world of $\mathcal{B}$ iff $S \subseteq F(\mathcal{B})$.*

Proposition 5 (No-observation completion boundary). *A single fixed fresh dispatch set both avoids duplicates and completes every obligation in every world of $\mathcal{B}$ iff $|\mathcal{B}| = 1$.*

Proof of Prop. 4. S is duplicate-free in world w iff $S \cap w = \emptyset$; for all $w \in \mathcal{B}$ iff $S \cap \bigcup \mathcal{B} = \emptyset$ iff $S \subseteq F(\mathcal{B})$. $\square$

Proof of Prop. 5. Completion in w requires $S \supseteq \Omega \setminus w$ and duplicate-freeness requires $S \cap w = \emptyset$, so $S = \Omega \setminus w$; one S serves two worlds only if they are equal. $\square$

Both statements are definitional once the belief family is fixed; E1 evaluates them exhaustively as a check of the artifact's code, not as evidence. Both depend on fresh mutation, no new observation and old-dispatch finality; a P_D sink within its window can replay retained keys without identifying the completed set.

B. Safety sufficiency is not query sufficiency

The ternary projection (done on D , absent on F , unknown on U) determines F exactly, so correlations never enlarge the immediate safe frontier. They change the cost of learning more. The v0.2 witness $\mathcal{B}_1 = \{\{a\}, \{b\}\}$ versus $\mathcal{B}_2 = 2^{\{a,b\}}$ has identical projections and optimal probe costs one and two.

C. Semantics-determined belief families

The source of correlation in practice is not exotic: it is the processing semantics of the request whose response was lost. Let a request carry ordered members $M = (\omega_1, \dots, \omega_n)$ and let the journal record M , its order and the provider's documented processing class.

Definition 3 (Classes). *independent: any subset may have committed ($\mathcal{B} = 2^M$; per-entry outcomes as in SQS batch and SES bulk [10], [11]). atomic: all or nothing ($\mathcal{B} = \{\emptyset, M\}$; a single-object request such as one Stripe charge [8]). prefix: ordered processing that stops at the first failure ($\mathcal{B} = \{\{\omega_1.. \omega_k\} : 0 \leq k \leq n\}$; the sequential worker of this artifact executing a batch wrapper is one instance). exact- k : a trustworthy aggregate reports k successes ($\mathcal{B} = \binom{M}{k}$).*

Proposition 6 (Journal sufficiency and probe costs). *For the independent, atomic and prefix classes, recording $(M, \text{order}, \text{class})$ before transport determines $\mathcal{B}$; the exact- k family additionally requires one post-hoc aggregate evidence call to learn k . Every class projects to all-unknown on M , so a ternary journal believes 2^M and needs $n = |M|$ unit probes. Under a uniform prior over worlds and unit-cost per-item probes, the exact optimal adaptive expected cost is n (independent), 1 (atomic), $\lceil \log_2 N \rceil - \frac{2^{\lceil \log_2 N \rceil} - N}{N}$ with $N = n+1$ (prefix), and $1 + n - \frac{n-k}{k+1} - \frac{k}{n-k+1}$ (exact- k , including the aggregate call).*

Proof. Determination is by construction of the families. Independent: a probe reveals one independent bit, so n probes are necessary and sufficient. Atomic: any single probe identifies the world. Prefix: worlds are the N prefixes ordered by length; probing ω_i splits the current candidate interval of lengths into the two contiguous sub-intervals $\{\geq i\}$ and $\{< i\}$, and conversely every split of a contiguous interval at a boundary is realized by probing the boundary item, so adaptive policies are exactly the binary decision trees over N ordered, equally likely leaves. Expected depth is minimized by the complete tree, whose leaves lie at depths $\lfloor \log_2 N \rfloor$ and $\lceil \log_2 N \rceil$ with $2^{\lceil \log_2 N \rceil} - N$ at the shallower depth. Exact- k : the v0.2 gap-symmetry argument with the count-aware stopping rule, plus the aggregate call. Under per-key finality the objective is no longer the probe count. A probe of an absent member is the fence that member needs before a fresh dispatch, so it adds no

further call, and a probe of a committed member is overhead. The optimum is then $(n+1)/2$ for the atomic class (any single probe), $n/2 + n/(n+1)$ for the prefix class (scan from the end of the request and stop at the first committed member) and n for the independent class. The probe-minimising prefix policy is the wrong policy for this objective: at $n = 8$ it costs 5.8 calls where the scan costs 4.9. A dynamic program over the belief family matches these closed forms for every $n \leq 10$ (independent through $n \leq 8$). The worst case under per-key finality is n for every class, the world in which nothing committed. $\square$

Two accountings of evidence cost are relevant. *Per-request finality*: the provider guarantees (by acknowledgement, SLA or a request-level finalize operation that returns no per-item outcome) that the lost request is finished; probes are then the only cost and fresh dispatch of certainly-absent items needs no further call. *Per-key finality*: the only final evidence is a per-key fence; every absent item must be fenced before its fresh dispatch, a probe counts as that item’s fence, and the cost is the quantity minimised in the proof. Table VII reports both. The class is a property the controller already reads from provider documentation to choose a profile; retaining it per request costs one field. Like T , it is trusted contract data: a misdeclared class makes the controller infer completion that did not happen.

VII. IMPLEMENTATION AND REPRODUCIBILITY

The v0.3 artifact is a fresh implementation written from the v0.2 manuscript rather than from its source tree, in Python standard library plus SQLite. Controller and provider run in different processes with different database files, WAL mode and `synchronous=FULL`. The controller stores immutable contracts, key/payload bindings, occurrence states with lease and retention deadline, receipts and events; insertion is transactional and (p, α) is unique. The provider inserts one row per effect atomically; a $P_D(T)$ dedup row carries an expiry; a P_F fence table is consulted inside the effect transaction so an in-flight request that arrives after the fence is rejected; every `/execute` call is logged with its outcome so attempts are distinguishable from effects. An evidence endpoint searches the retained effect table, modelling providers whose history outlives their idempotency window; it can be disabled.

Two policies share transport and persistence. R performs flat canonical-action matching and consults the ledger per occurrence; V checks typed wrapper membership on the whole plan and residualizes. Three recovery modes exist for a held occurrence: *naive* (same-key retry regardless of profile or retention); *lookup-then-fresh* (query the evidence endpoint and, on a negative answer, dispatch under a fresh key without fencing, which is a hand-written handler); a durable-execution retry keyed by activity identity is the naive policy); and *aware* (Def. 2). Ledger writes that confirm or abandon an occurrence are accepted only from the current lease holder, so a predecessor that resumes after a takeover cannot overwrite the successor’s state. Two exceptions are

studied in I6. A receipt whose key and payload hash bind to an occurrence already *unresolved* may be adopted (*adopt*) or refused (*reject*). A binding-matched receipt that arrives while the successor still holds the occurrence is stored and adopted if the holder later gives up, and dropped if the holder rekeys, because the key no longer matches. Three invariants carry this: `reserve` refuses *unresolved*, so the controller never fresh-dispatches from that state; `rekey` requires the holder; adoption requires the current binding. The worker’s fault action is either SIGKILL or SIGSTOP; a stopped worker is resumed by the orchestrator with SIGCONT. For the evidence study the worker can send a wrapper group as one multi-entry request; the provider processes entries by a declared class (independent, atomic, prefix) with a per-entry failure hook, and the controller journals membership, order and class before transport. A recovering worker with a *ternary* journal probes every held member; with a *structured* journal it rebuilds the belief family of Section VI and probes by the exact oracle. The probe is either a point-in-time lookup (per-request finality supplied by the study’s timing) or a per-key fence (per-key finality), and in the latter accounting every absent member is fenced before its fresh dispatch. The successor may also be started with a journal class that contradicts the provider’s actual processing, to measure the trusted-contract failure mode. Fault points terminate the process with a real SIGKILL after the first reservation, after the first effect before confirmation, after the first confirmation, after the third effect, or during the first transport (a timer kills the process 150 ms after the request is sent while the provider delays commit).

The artifact contains 37 kernel unit tests, all experiment drivers, raw per-trial records (635 process trials with return codes, crash markers, successor summaries and provider-side counts), exhaustive enumerations and model-checking witnesses. `run_all.py` regenerates every number in this paper in about fifteen minutes on one core. Manifests are local provenance, not external timestamping.

VIII. STUDY DESIGN

D1 and D1-TTL. For $n = 1..4$, enumerate all $n!2^{n-1}$ ordered groupings as source and destination and every committed prefix of the source order: $\sum_{n=1}^4 (n!2^{n-1})^2(n+1) = 186,674$ cases. Policies: G (group-scoped key), R , R^+ , V , V_r (retention-aware, no evidence), V_r+E (retention-aware with lookup). Six variants cross the state of the last committed occurrence at the crash—durably confirmed, committed with lost acknowledgement, or reserved without effect—with whether the dedup index was reclaimed before recovery.

M1. One obligation, two key generations, two senders per generation, one crash, plus a retention-expiry transition that fires only while no request is in flight. Breadth-first search exhausts the safe configuration; each mutant stops at its shortest violating trace. State counts are specific to this encoding and not comparable to v0.2’s.

I1, I2. As in v0.2: 2 policies $\times$ 6 source/destination grouping pairs $\times$ 4 fault points (after the first reservation, after the first effect, after the first confirmation, and after the

third effect) $\times 5$ repetitions = 240 SIGKILL/recovery trials on P_D . The during-transport fault is used by I3, I5 and I6 rather than by I1. I2 is 12 three-worker races per policy. The runtime policy labeled R consults the ledger, so it is R^+ of Table I.

I3. Source plan one batch of four; the worker is killed 150 ms into its first transport while the provider delays commit by 1000 ms (*early*) or 200 ms (*late*). A different worker adopts the held occurrence after lease expiry at 0.5 s or 0.9 s. 3 profiles $\times 2$ variants $\times 3$ recovery modes $\times 5$ repetitions = 90 trials. The evidence endpoint is enabled for every profile so that lookup-then-fresh has the information it would have in practice.

I4. $P_D(T)$ with $T = 0.5$ s. The worker is killed after the first effect (lost acknowledgement) or after the first reservation (no effect); the successor starts 0.8 s later. 2 evidence settings $\times 2$ faults $\times 2$ recovery modes $\times 2$ policies $\times 5 = 80$ trials.

I6. As I3, but the predecessor is SIGSTOPped after its first reservation or 150 ms into its first transport (provider commit delay 1000 ms), the successor recovers with the aware policy at 0.5 s, and the predecessor is resumed at 1.6 s and runs to completion. 3 profiles $\times 2$ stop points $\times 2$ late-evidence rules $\times 5 = 60$ trials.

I5. One batch request over n members is sent and the worker is killed before it can read the response; the provider fails the entries designated by the world. Every world of each class is run once: independent $n=4$ (16 worlds), atomic $n=4, 8$ (2 each), prefix $n=4, 8$ (5 and 9). The successor starts after the provider finished (finality by construction), reconciles from a ternary or a structured journal, confirms committed members and fresh-dispatches absent ones under new keys on P_F . There are 34 worlds (16 independent, 2+2 atomic, 5+9 prefix). Each is run under both journals and both accountings (136 trials); 5 further prefix worlds are run with the journal misdeclaring the class as atomic. 141 trials. Under per-key finality the structured journal uses the fence-minimising policy of Prop. 6, which is not the probe-minimising policy.

E1–E4. E1 enumerates all 65,535 belief families on four occurrences against all 16 dispatch sets. E2 is the correlation witness. E3 compares the exact- k closed form with an exact dynamic-programming oracle for all $0 < k < n \leq 10$ (2,026 worlds; v0.2 stopped the oracle at $n = 8$). E4 computes ternary and structured optimal probe costs for the independent, atomic and prefix classes.

The success oracle for every process trial requires the crashed worker to exit with -9 , the successor with 0, and reads the provider’s effect table independently of controller state.

IX. RESULTS

A. The reduction holds and the strengthened reference ties

Table I reproduces the v0.2 counts exactly (16 / 1,308 / 129,024 group-key duplicates) from an independent implementation. R^+ and V agree on every effect vector and make the same 372,145 calls; R makes twice as many because it resends completed occurrences and lets the sink deduplicate.

TABLE I

D1: EXHAUSTIVE REGROUPING ON P_D WITH UNBOUNDED RETENTION. DUPLICATES ARE CASE COUNTS, NOT AGENT FAILURE RATES. R^+ AND V HAVE IDENTICAL EFFECT VECTORS IN EVERY CASE AND IDENTICAL CALL COUNTS.

n	Plans	Cases	G dup	R dup	R^+ dup	V dup
1	1	2	0	0	0	0
2	4	48	16	0	0	0
3	24	2,304	1,308	0	0	0
4	192	184,320	129,024	0	0	0
Total		186,674	130,348	0	0	0
Sink calls			744,290	744,290	372,145	372,145

TABLE II

D1-TTL: THE SAME 186,674 CASES WHEN THE DEDUP INDEX IS RECLAIMED BETWEEN CRASH AND RECOVERY. *dup*: CASES WITH A DUPLICATED OCCURRENCE; *unres*: CASES THE CONTROLLER LEAVES UNRESOLVED; *inc*: CASES WHERE AN APPROVED OCCURRENCE NEVER HAPPENED.

Last occ. at crash	Policy	dup	unres	inc	calls
confirmed	R	149,217	0	0	744,290
	R^+, V	0	0	0	372,145
	$V_r, V_r + E$	0	0	0	372,145
lost ack	R	149,217	0	0	744,290
	R^+, V	149,217	0	0	521,362
	V_r	0	149,217	0	372,145
	$V_r + E$	0	0	0	521,362
reserved, no effect	R	111,760	0	0	744,290
	R^+, V	0	0	0	521,362
	V_r	0	149,217	149,217	372,145
	$V_r + E$	0	149,217	149,217	521,362

In I1 all 240 trials pass for both policies; the provider logs 1,080 execute calls for 960 effects, and the 120 excess calls are exactly the lost-acknowledgement retries (60 trials each at the after-first-effect and after-third-effect fault points) answered with a cached receipt. In I2 all 24 races pass; the provider logs 96 calls, all committed, none deduplicated—controller reservation resolves every race before the sink is reached. Median successor wall time is 0.101 s (R) and 0.102 s (V) including interpreter start; it is descriptive only.

B. After the window, every same-key policy duplicates

Table II is the finite-domain form of Prop. 3. With reclaimed retention and a lost acknowledgement, R , R^+ and V all duplicate in every one of the 149,217 cases with a nonempty committed prefix. The verifier’s plan-level residualization protects only occurrences the controller already recorded as done. The retention-aware policy is safe in every variant; without evidence it pays with 149,217 unresolved cases, which are false alarms under lost acknowledgement and real gaps when nothing had committed. With a lookup it completes every lost-acknowledgement case at one evidence call per lapsed occurrence, but in the reserved-without-effect variant its negative lookup is not finality evidence, so it stops at *unresolved* exactly as V_r does and has paid one more call to learn nothing it could act on; this is the abstract counterpart of the “after reservation,

TABLE III

M1 BOUNDED REACHABILITY. ONLY SAFE ROWS EXHAUST THEIR GRAPH; MUTANTS STOP AT THE SHORTEST WITNESS.

Model	Configuration	States	Edges	Witness
v0.2	safe	1,180	3,467	none (exhausted)
	absence without fence	445	1,036	9 (duplicate)
	fresh-key retry	395	939	7 (duplicate)
	missing as success	11	11	2 (false success)
	no sink dedup	65	126	6 (duplicate)
+retention	safe	1,396	4,043	none (exhausted)
	retry after expiry	121	249	7 (duplicate)
	safe, no fence capability	76	158	none (exhausted)

aware, lookup” row of Table VI. Without retention expiry, all six policies except G are safe and complete in all three crash-state variants. We stress that these counts are a consequence of the proposition applied to each case, not independent evidence for it: the enumeration checks that the implementation of V_r behaves as specified on every regrouping, which is its purpose.

C. Model checking

The retention mutant’s witness is: hold, send, commit, crash, retention expires, resend same key, commit. The safe configuration with the retention transition still exhausts without violation because its send rule refuses any key that may have been dispatched before expiry; recovery proceeds through fence, lookup and certificate instead. The last row removes the fence transition to model a $P_D(T)$ sink that offers no fence: the safe controller still exhausts without a duplicate or false success, in a much smaller graph, because after expiry its only remaining move for a possibly-dispatched key is to stop. M1 checks safety only; it does not check liveness, and that row is precisely the configuration in which the controller can be safe by never completing.

D. Takeover with an in-flight request

Table IV separates the mechanisms. Under P_D the sink absorbs a same-key race in both orders. Under P_F naive retry duplicates every time; the fence either rejects the in-flight request and licenses a fresh key or returns the committed receipt. Under P_O the only safe early action is to stop: the successor’s lookup is negative because the old request has not landed yet, and 0.5 s later the world is in fact complete while the controller correctly reports one unresolved occurrence. The naive controller “completes” every P_F and P_O trial with a duplicate. The controller never learns which happened without the provider’s effect table, which is why the oracle reads it directly.

The lookup-then-fresh-key rows are a hand-written handler we expect to meet in practice: it queries the provider and, seeing nothing, re-issues under a new idempotency key. They are not what Temporal prescribes. A durable-execution retry whose key is the activity identity is the naive row, and its failure mode is the expired window of Table VI, not this one. It is correct in every *late* cell, where the lookup is positive, and duplicates in every *early* cell on every profile—including

TABLE IV

I3: A SUCCESSOR ADOPTS A HELD OCCURRENCE WHILE THE CRASHED WORKER’S REQUEST IS STILL IN FLIGHT (*early*) OR AFTER IT COMMITTED (*late*). CELLS ARE COUNTS OUT OF 5. *naive*: SAME-KEY RETRY; $L \rightarrow F$: NEGATIVE LOOKUP THEN FRESH KEY, NO FENCE; *aware*: DEF. 2. *dup*: PROVIDER RECORDED TWO EFFECTS; *done*: CONTROLLER FINISHED ALL FOUR; *unres*: CONTROLLER LEFT ONE UNRESOLVED.

Profile	Variant	Recovery	dup	done	unres	Mechanism
P_D	early	naive	0	5	0	sink dedup
P_D	early	$L \rightarrow F$	5	5	0	<i>none</i> : two keys, two effects
P_D	early	aware	0	5	0	sink dedup
P_D	late	naive	0	5	0	cached receipt
P_D	late	$L \rightarrow F$	0	5	0	positive lookup
P_D	late	aware	0	5	0	cached receipt
P_F	early	naive	5	5	0	<i>none</i>
P_F	early	$L \rightarrow F$	5	5	0	<i>none</i>
P_F	early	aware	0	5	0	fence rejects old, fresh key
P_F	late	naive	5	5	0	<i>none</i>
P_F	late	$L \rightarrow F$	0	5	0	positive lookup
P_F	late	aware	0	5	0	fence returns receipt
P_O	early	naive	5	5	0	<i>none</i>
P_O	early	$L \rightarrow F$	5	5	0	<i>none</i>
P_O	early	aware	0	0	5	unresolved (world done)
P_O	late	naive	5	5	0	<i>none</i>
P_O	late	$L \rightarrow F$	0	5	0	positive lookup
P_O	late	aware	0	5	0	positive lookup

TABLE V

I6: THE PREDECESSOR IS SUSPENDED (SIGSTOP) RATHER THAN KILLED, OVERTAKEN BY AN AWARE SUCCESSOR, THEN RESUMED (SIGCONT) WITH A STALE LEASE. COUNTS OUT OF 5 PER MODE; ROWS MARKED *reject* / *adopt* ARE IDENTICAL IN BOTH LATE-EVIDENCE MODES. *Zombie write*: WHAT HAPPENED TO THE RESUMED WORKER’S REQUEST OR RECEIPT.

Profile	Stopped	Late evid.	dup	done	unres	Zombie write
P_D	reserved	reject / adopt	0	5	0	receipt (dedup) refused, 5
P_D	in flight	reject / adopt	0	5	0	receipt (dedup) refused, 5
P_F	reserved	reject / adopt	0	5	0	fence rejected request, 5
P_F	in flight	reject / adopt	0	5	0	fence rejected request, 5
P_O	reserved	reject	0	0	5	receipt refused, 5
P_O	reserved	adopt	0	5	0	receipt adopted, 5
P_O	in flight	reject	0	0	5	receipt refused, 5
P_O	in flight	adopt	0	5	0	receipt adopted, 5

P_D . Sink deduplication cannot help there because the fresh key is a different key; a negative point-in-time lookup while the old request is still in flight is precisely the non-evidence of Def. 2, and the fence is the only operation in the model that converts it into finality. This is the runtime instance of the nine-step “absence without fence” witness of Table III.

E. Resumed predecessor

A crashed worker is the easy case; a worker that was merely paused—by the scheduler, a debugger, a VM migration or a long garbage-collection pause—resumes with a lease it no longer holds and a request that may already have been answered. I6 makes this concrete: the predecessor is stopped either after reserving its first occurrence (request not yet sent) or 150 ms into the transport (request in flight), the successor recovers with Def. 2, and the predecessor is then resumed and allowed to run to completion (Table V).

TABLE VI

I4: RECOVERY 0.8 S AFTER A CRASH ON A $P_D(T)$ SINK WITH $T = 0.5$ S. COUNTS OUT OF 5; R AND V ARE IDENTICAL IN EVERY CELL AND ARE MERGED.

Evidence	Crash point	Recovery	dup	done	unres
lookup	after first effect	naive	5	5	0
lookup	after first effect	aware	0	5	0
lookup	after reservation	naive	0	5	0
lookup	after reservation	aware	0	0	5
none	after first effect	naive	5	5	0
none	after first effect	aware	0	0	5
none	after reservation	naive	0	5	0
none	after reservation	aware	0	0	5

Three observations follow. First, the ledger’s lease-holder rule does what it is for. Wherever the occurrence was *held* or *done*, the zombie’s state write was refused and journaled; the ten P_O trials under the adopt rule are the exception, and there the write is taken as evidence rather than as a claim to the lease. In every trial the zombie completed the remaining occurrences by reading the successor’s *done* records rather than re-executing them. Second, duplicate-freedom does not come from that rule. Under P_D the zombie’s late same-key request is deduplicated by the sink; under P_F the successor’s fence rejects it at the provider; under P_O nothing rejects it, but the successor—correctly—never dispatched a fresh key, so the zombie’s request is the only effect. The zombie is a very late in-flight request, and the mechanisms of Table IV govern it. Third, under P_O the zombie carries the only finality evidence that exists: a receipt bound to the exact key and payload hash of an occurrence the successor had to leave *unresolved*. A ledger that treats every non-holder write as a stale claim discards that evidence and stays incomplete. Separating the two questions—*who may transition state* (the lease holder) from *what counts as evidence* (a receipt whose binding matches)—lets an *unresolved* occurrence adopt a late receipt while *held* and *done* occurrences still refuse non-holder writes. Safety does not rest on “no other evidence.” It rests on three invariants, each enforced by the ledger and covered by a unit test: a reservation is refused once the state is *unresolved*, so no fresh key is ever dispatched from it; only the holder can rekey; and a late receipt is adopted only when its key and payload hash equal the occurrence’s current binding, which fails after a rekey. A receipt that arrives while the successor still holds the occurrence is stored and adopted at the moment the holder would otherwise give up, which closes the race in which the holder marks *unresolved* after refusing the zombie. I6 itself resumes the zombie only after the successor has finished; the during-recovery interleaving is the unit test, not the process study. With that rule the P_O trials complete; with the strict rule they remain unresolved. Neither rule affects the effect count.

F. Retention expiry at runtime

Naive same-key retry after the window duplicates in 20 of 20 lost-acknowledgement trials, for both policies. The retention-aware controller never duplicates. With a lookup it

TABLE VII

E4: EXACT OPTIMAL EXPECTED EVIDENCE CALLS FOR A LOST REQUEST OVER n MEMBERS, UNIFORM PRIOR. *Probes* ASSUME PER-REQUEST FINALITY; $+fence$ IS THE EXPECTED COST OF THE POLICY THAT MINIMISES PROBES PLUS THE FENCES OF MEMBERS NOT YET PROBED. THE TERNARY JOURNAL COSTS n IN EVERY COLUMN AND CLASS; THE INDEPENDENT CLASS COSTS n UNDER EVERY ACCOUNTING.

n	Probes			$+fence$ (exp.)		Exact- $\lfloor n/2 \rfloor$	
	Atomic	Prefix	Pref. worst	Atomic	Prefix	Probes	Ternary
2	1	1.667		2	1.500	1.667	1.000
4	1	2.400		3	2.500	2.800	2.667
6	1	2.857		3	3.500	3.857	4.500
8	1	3.222		4	4.500	4.889	6.400
10	1	3.545		4	5.500	5.909	8.333

completes all lost-acknowledgement trials; without one, or when nothing had committed, it stops at *unresolved* because a negative point-in-time lookup is not finality evidence. The “after reservation, aware, lookup” row is the price of correctness under $P_D(T)$ without a fence: the fresh dispatch would have been safe, but the controller cannot know it.

G. Evidence contracts

E1 performs 1,048,560 checks with no mismatch against the frontier rule or the singleton characterization; 941 of 65,535 families admit a nonempty safe dispatch. E2 confirms costs one and two for identical ternary snapshots. E3 agrees with the exact- k closed form in all 45 (n, k) pairs and 2,026 worlds, and the exact oracle matches it for every $n \leq 10$. Table VII shows the DP oracle and the closed forms agreeing for every n . Under per-request finality the prefix class is the strongest realistic case for the journal contract: with $n = 8$ the structured journal needs 3.2 probes on average and 4 in the worst case, against 8 for the ternary journal, and the atomic class needs one. Under per-key finality the structured journal still helps, and by more than a probe-minimising policy would: 4.5 and 4.9 calls of 8 for atomic and prefix, against 8 for the ternary journal, with a worst case of n when nothing committed. The saving is exactly the committed members the scan does not probe. A single call that returns every item status at cost c_L beats per-item probing whenever c_L is below the probe count; the structured journal beats that call when c_L exceeds the structured cost itself (3.2 for prefix at $n = 8$ under per-request finality), not the gap between the two journals. The exact- k column counts per-item probes after the aggregate call that learns k ; the total in Prop. 6 adds that call.

Table VIII closes the loop between the mathematics and the implementation: the structured journal’s measured probe counts equal the oracle expectation in every cell, the ternary journal pays n in every cell, and both are exactly-once and complete in all 136 trials because absent members are dispatched under fresh keys only after the world is identified. Under the per-key-fence accounting the totals again match Table VII exactly, including the 4.5 and 4.9 of 8 for atomic and prefix. The probe column is the per-request policy; the call column is the other policy, and the two are not the same set of probes. The journal costs one field per request.

TABLE VIII

I5: RUNTIME RECONCILIATION OF A LOST MULTI-ENTRY RESPONSE, EVERY WORLD OF EACH CLASS RUN ONCE UNDER TWO ACCOUNTINGS. *Probes*: MEAN DECISION PROBES UNDER PER-REQUEST FINALITY. *Calls*: MEAN TOTAL EVIDENCE CALLS UNDER PER-KEY FINALITY, USING THE FENCE-MINIMISING POLICY, WHICH PROBES A DIFFERENT SET. ORACLE COLUMNS ARE THE E4 VALUES. ALL 136 TRIALS ARE EXACTLY-ONCE AND COMPLETE.

Class	n	Journal	Worlds	Probes		+fence		
				Mean	Max	Oracle	Calls	Oracle
indep.	4	ternary	16	4.00	4	4.00	4.00	4.00
indep.	4	structured	16	4.00	4	4.00	4.00	4.00
atomic	4	ternary	2	4.00	4	4.00	4.00	4.00
atomic	4	structured	2	1.00	1	1.00	2.50	2.50
prefix	4	ternary	5	4.00	4	4.00	4.00	4.00
prefix	4	structured	5	2.40	3	2.40	2.80	2.80
prefix	8	ternary	9	8.00	8	8.00	8.00	8.00
prefix	8	structured	9	3.22	4	3.22	4.89	4.89
atomic	8	ternary	2	8.00	8	8.00	8.00	8.00
atomic	8	structured	2	1.00	1	1.00	4.50	4.50

The misdeclaration trials show the trusted-contract failure mode. Five prefix worlds at $n = 4$ were reconciled by a journal that claimed the request was atomic. The controller probed one member, inferred the rest and reported all four obligations done in all five trials; in three of them (the worlds where processing stopped after one, two or three entries) the provider’s table shows the remaining obligations never happened. No duplicate occurred, because in the one world where the probe was negative nothing had committed and fresh dispatch of everything was correct. False completion, not duplication, is the failure a wrong class produces here; a wrong class in the other direction (an independent request declared prefix) would produce the opposite, a fresh dispatch of an item that had committed.

X. DISCUSSION AND THREATS TO VALIDITY

A. What changed

The v0.2 null result stands and is now reproduced by a second implementation with a stronger reference. The contribution is no longer “verified refinement” but the precise boundary of the reduction and what lies beyond it. Two boundary conditions are documented provider facts, not constructions: finite retention windows and vendor-acknowledged opaque outcomes. On the retention side the necessary controller knowledge is minimal and already durable ($h(\omega)$ and T), and the correct behaviour after the window is a profile downgrade, not a heuristic. On the evidence side the necessary knowledge is one field per request (the processing class), and its value is exact and class-dependent.

For an agent runtime this means: a tool adapter must declare T , the evidence capability and the batch class as contract fields; a controller that stores only “unknown outcome” and never retries is safe under P_O but forfeits completion that a lookup would have restored; a controller that retries with the same idempotency key without checking $h(\omega) + T$ is not exactly-once against Stripe or SQS FIFO after an outage longer than the window; and a controller that looks up the

outcome and re-issues under a fresh key on a negative answer is not exactly-once while the old request can still land, on any profile, because deduplication is keyed and the fresh key is new. The last point is the one a hand-written recovery handler reaches. A durable-execution engine that reuses the activity’s key is the naive row, and it is exactly-once only inside the window of Prop. 3.

B. Internal and construct validity

The same process designed policies, workloads and oracles; the provider’s independent effect table reduces but does not remove design bias. Reference and verifier share storage and transport; a common bug could affect both, which is why the exhaustive abstract study exists alongside the runtime. The finite model abstracts torn writes and packet corruption. Exactly-once is measured at the synthetic insertion boundary. The P_F fence and the evidence endpoint are our own implementations of documented capabilities, not vendor endpoints. I3 and I4 depend on sleeps and are deterministic in this environment across 5 repetitions per cell; they are demonstrations of mechanism, not measurements of frequency.

Several results are true by construction and are reported as checks rather than findings. The naive baseline duplicates after the window because it is defined to retry the same key; its value is to show that the v0.2 reference and verifier *are* that baseline once retention lapses. The D1-TTL counts follow from Prop. 3 case by case. Lemma 1 makes the R^+/V tie structural. In I5 the finality of the lost request is supplied by the schedule (the successor starts after the provider finished); under the per-key accounting the fence supplies it instead, and the two accountings bracket what a real provider would offer. The processing class and T are read from a trusted contract, and the misdeclaration row shows the consequence of getting them wrong. I6 exercises a suspended, overtaken and resumed predecessor at the process level, but with a cooperative pause: the zombie is resumed only after the successor has finished. A zombie resumed *during* the successor’s recovery races the successor’s fence or rekey at the ledger; the lease-holder rule serializes the ledger side of that race inside one SQLite transaction, but the provider side is again the I3 *early* case and rests on fence or dedup, not on the ledger. The late-evidence adoption rule was designed after observing the strict rule’s result and is reported as a design consequence with its own measurement, not as a pre-registered hypothesis.

C. External and statistical validity

There are no live LLM traces; the frequency with which agents regroup or replan is unmeasured. Retention windows are quoted from documentation current on the access date and can change. The prefix class is grounded in this artifact’s own sequential worker and in ordered stop-on-failure processors generally; we did not verify a specific vendor batch API with that semantics. Exhaustive counts do not justify confidence intervals about deployments; we report exact counts only.

D. Evidence and authorship boundaries

All numbers derive from executed programs in the artifact. No paid inference, production mutation or public submission occurred. Generative-AI assistance was used for code, mathematics and text; human review of proofs, citations, code and authorship is required before submission. Characterizations of [1], [2], [3], [4], [6] are carried from the v0.2 literature audit and must be re-verified against the current versions of those preprints.

XI. NEXT EVALUATION GATE

The retention hazard is now specific enough to test against a real contract without spending on effects: a sandbox account for a $P_D(T)$ provider can confirm the documented post-window behaviour with test-mode objects, and the controller field set (T , evidence, class) can be validated against three adapters with different profiles. The evidence-contract claim needs one provider with a per-request processing class other than independent, and a comparison that grants the structured journal's information to a strong baseline. Natural LLM proposals should be collected separately from stress transformations. If both gates pass, the runtime contract fields become the deliverable; if they fail, this remains a reproducibility and negative-result study with a documented hazard.

XII. CONCLUSION

Stable obligation keys are sufficient for exactly-once effects of re-planning agents exactly as long as the sink retains them, and a second implementation confirms that a verifier adds nothing under that hypothesis. Commercial sinks bound retention. Beyond the bound, the same policies duplicate in every lost-acknowledgement case, the safe decision depends only on the controller's own hold time and the documented window, and completion depends on an evidence contract whose cost is determined by what the journal retained about the lost request. The artifact supplies exact finite-domain, model-checked and process-level evidence for each of these statements and states the conditions under which each stops applying.

REFERENCES

- [1] Y. Zheng, Y. Yang, W. Zhang, and A. Quinn, "ACRFence: Preventing semantic rollback attacks in agent checkpoint-restore," arXiv:2603.20625, 2026.
- [2] J. Xu, L. Fan, Z. Wang, X. Li, and H. Liu, "Beyond single-use tokens: Durable authorization state for replay-resistant LLM agent actions," arXiv:2608.01710, 2026.
- [3] Y. Tong, L. Dai, and S. Guo, "AID-Guard: Stateful authorization for delegated agent effects," arXiv:2608.21159, 2026.
- [4] Z. Chen et al., "Cordon: Semantic transactions for tool-using LLM agents," arXiv:2606.17573, 2026.
- [5] C. Lee, S. J. Park, A. Kejriwal, S. Matsushita, and J. Ousterhout, "Implementing linearizability at large scale and low latency," in *Proc. SOSP*, 2015, pp. 71–86.
- [6] Z. Wang, "Proof-carrying agent actions: Model-agnostic runtime governance for heterogeneous agent systems," arXiv:2606.04104, 2026.
- [7] Temporal Technologies, "What is a Temporal Activity?" Official documentation, accessed 19 September 2026. <https://docs.temporal.io/activities>
- [8] Stripe, "Idempotent requests," API reference, accessed 19 September 2026. https://docs.stripe.com/api/idempotent_requests
- [9] Amazon Web Services, "Using the message deduplication ID in Amazon SQS," Developer Guide, accessed 19 September 2026. <https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/using-messagededuplicationid-property.html>
- [10] Amazon Web Services, "SendMessageBatch—Amazon SQS API Reference," accessed 19 September 2026. https://docs.aws.amazon.com/AWSSimpleQueueService/latest/APIReference/API_SendMessageBatch.html
- [11] Amazon Web Services, "SendBulkEmail—Amazon SES API v2 Reference," accessed 19 September 2026. https://docs.aws.amazon.com/ses/latest/APIReference-V2/API_SendBulkEmail.html
- [12] Slack, "conversations.history," Web API method reference, accessed 19 September 2026. <https://api.slack.com/methods/conversations.history>
- [13] Slack, "chat.postMessage," Web API method reference, accessed 19 September 2026. <https://api.slack.com/methods/chat.postMessage>
- [14] R. I. Brafman and G. Shani, "Replanning in domains with partial information and sensing actions," *J. Artif. Intell. Res.*, vol. 45, pp. 565–600, 2012.
- [15] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, "Planning and acting in partially observable stochastic domains," *Artif. Intell.*, vol. 101, no. 1–2, pp. 99–134, 1998.
- [16] P. Helland, "Life beyond distributed transactions: An apostate's opinion," in *Proc. CIDR*, 2007, pp. 132–141.
- [17] P. Helland, "Idempotence is not a medical condition," *Commun. ACM*, vol. 55, no. 5, pp. 56–65, 2012.
- [18] H. Garcia-Molina and K. Salem, "Sagas," in *Proc. ACM SIGMOD*, 1987, pp. 249–259.
- [19] P. Carbone, S. Ewen, G. Fóra, S. Haridi, S. Richter, and K. Tzoumas, "State management in Apache Flink: Consistent stateful distributed stream processing," *Proc. VLDB Endow.*, vol. 10, no. 12, pp. 1718–1729, 2017.
- [20] G. Wang, L. Chen, A. Dikshit, J. Gustafson, B. Chen, M. J. Sax, J. Roesler, S. Blee-Goldman, B. Cadonna, A. Mehta, V. Madan, and J. Rao, "Consistency and completeness: Rethinking distributed stream processing in Apache Kafka," in *Proc. ACM SIGMOD*, 2021, pp. 2602–2613.
- [21] J. Jena and S. Dalal, "The Idempotency-Key HTTP header field," IETF Internet-Draft draft-ietf-httpapi-idempotency-key-header, work in progress, accessed 19 September 2026.
- [22] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.
- [23] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Proc. NeurIPS*, 2023.
- [24] G. J. Holzmann, "The model checker SPIN," *IEEE Trans. Softw. Eng.*, vol. 23, no. 5, pp. 279–295, 1997.
- [25] A. D. Birrell and B. J. Nelson, "Implementing remote procedure calls," *ACM Comput. Syst.*, vol. 2, no. 1, pp. 39–59, 1984.
- [26] B. W. Lampson, "Reliable messages and connection establishment," in *Distributed Systems*, S. Mullender, Ed., 2nd ed. ACM Press, 1993.
- [27] B. Liskov, L. Shrira, and J. Wroclawski, "Efficient at-most-once messages based on synchronized clocks," *ACM Trans. Comput. Syst.*, vol. 9, no. 2, pp. 125–142, 1991.
- [28] C. Gray and D. Cheriton, "Leases: An efficient fault-tolerant mechanism for distributed file cache consistency," in *Proc. SOSP*, 1989, pp. 202–210.
- [29] M. Burrows, "The Chubby lock service for loosely-coupled distributed systems," in *Proc. OSDI*, 2006, pp. 335–350.
- [30] G. Ramalingam and K. Vaswani, "Fault tolerance via idempotence," in *Proc. POPL*, 2013, pp. 249–262.
- [31] S. Setty, C. Su, J. R. Lorch, L. Zhou, H. Chen, P. Patel, and J. Ren, "Realizing the fault-tolerance promise of cloud storage using locks with intent," in *Proc. OSDI*, 2016.
- [32] H. Zhang, A. Cardoza, P. B. Chen, S. Angel, and V. Liu, "Fault-tolerant and transactional stateful serverless workflows," in *Proc. OSDI*, 2020, pp. 1187–1204.
- [33] Z. Jia and E. Witchel, "Boki: Stateful serverless computing with shared logs," in *Proc. SOSP*, 2021, pp. 691–707.
- [34] S. Burckhardt, C. Gillum, D. Justo, K. Kallas, C. McMahon, and C. S. Meiklejohn, "Durable functions: Semantics for stateful serverless," *Proc. ACM Program. Lang.*, vol. 5, no. OOPSLA, pp. 1–27, 2021.
- [35] L. Hyafil and R. L. Rivest, "Constructing optimal binary decision trees is NP-complete," *Inf. Process. Lett.*, vol. 5, no. 1, pp. 15–17, 1976.

- [36] SQLite, “Atomic commit in SQLite” and “Write-ahead logging,” official documentation, accessed 19 September 2026. <https://sqlite.org/atomiccommit.html>